ANSIBLE CHALLENGE

📘 CI/CD PIPELINE PROJECT DOCUMENTATION

Terraform + Ansible + Jenkins (AWS)

1️⃣ Project Overview

This project automates infrastructure provisioning and configuration using a CI/CD pipeline.

- Terraform → creates AWS infrastructure
- Ansible → configures servers
- Jenkins → orchestrates the full pipeline

2️⃣ Files & Directories Created (IMPORTANT SECTION)

📁 Terraform Files

File	Purpose
main.tf	Defines AWS EC2 instances (frontend & backend)
variables.tf	Stores reusable variables
output.tf	Outputs instance details
inventory.tpl	Template to generate Ansible inventory dynamically

---

📁 Ansible Files

File	Purpose
inventory	Dynamic inventory generated by Terraform
site.yml	Main Ansible playbook
roles/common/tasks/main.yml	OS-level configurations
roles/frontend/tasks/main.yml	Nginx installation & config
roles/frontend/templates/nginx.conf.j2	Nginx reverse proxy template
roles/backend/tasks/main.yml	Netdata installation

📁 Jenkins File

File	Purpose
Jenkinsfile	CI/CD pipeline definition

3️⃣ Project Directory Structure

```
ansible_project/
├──── CI_pipeline/
│     ├──── Terraform/
│     │     ├──── main.tf
│     │     ├──── variables.tf
│     │     ├──── output.tf
│     │     └──── inventory.tpl
│     │
│     └──── Ansible/
```

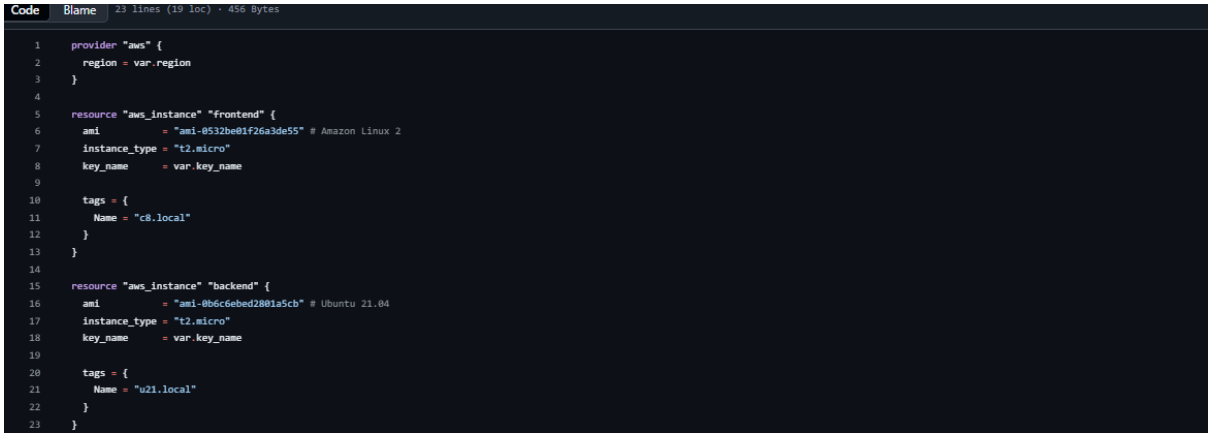
```
│       ├── inventory
│       ├── site.yml
│       └── roles/
│           ├── common/
│           │   └── tasks/main.yml
│           ├── frontend/
│           │   ├── tasks/main.yml
│           │   └── templates/nginx.conf.j2
│           └── backend/
│               └── tasks/main.yml
│
└── Jenkinsfile
```

4️⃣ Terraform Configuration Explained

🔹 main.tf

- Creates 2 EC2 instances
  - Amazon Linux → frontend
  - Ubuntu → backend
- Assigns:
  - Security groups
  - Key pair
  - Tags & hostnames
- Code:

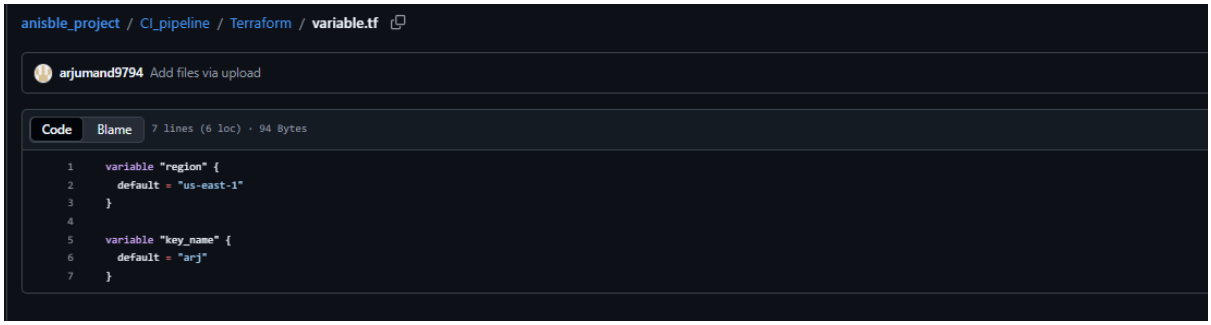
```
provider "aws" {
  region = var.region
}

resource "aws_instance" "frontend" {
  ami           = "ami-0532be01f26a3de55" # Amazon Linux 2
  instance_type = "t2.micro"
  key_name      = var.key_name

  tags = {
    Name = "c8.local"
  }
}

resource "aws_instance" "backend" {
  ami           = "ami-0b6c6ebed2801a5cb" # Ubuntu 21.04
  instance_type = "t2.micro"
  key_name      = var.key_name

  tags = {
    Name = "u21.local"
  }
}
```



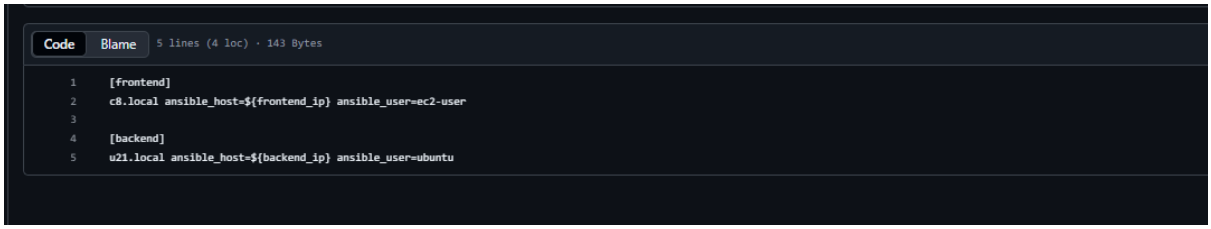
- **variables.tf**
  - Stores:
    - AWS region
    - AMI IDs
    - Instance types
  - Improves reusability and readability

Code:

```
variable "region" {
  default = "us-east-1"
}


variable "key_name" {
  default = "arj"
}
```



- **inventory.tpl**
  - Used to dynamically generate Ansible inventory
  - Example output:



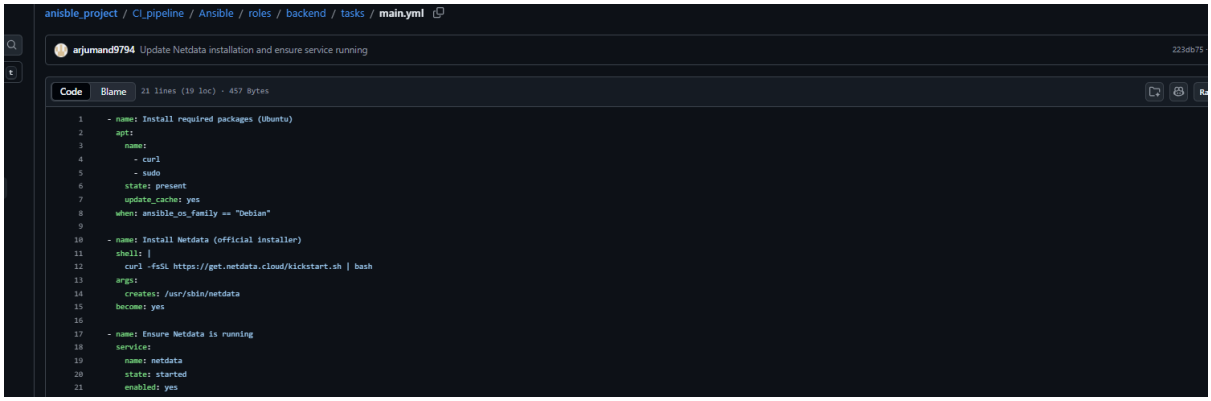
[frontend]

c8.local ansible_host=<frontend-ip> ansible_user=ec2-user

[backend]

u21.local ansible_host=<backend-ip> ansible_user=ubuntu

✔ Eliminates manual inventory updates



---

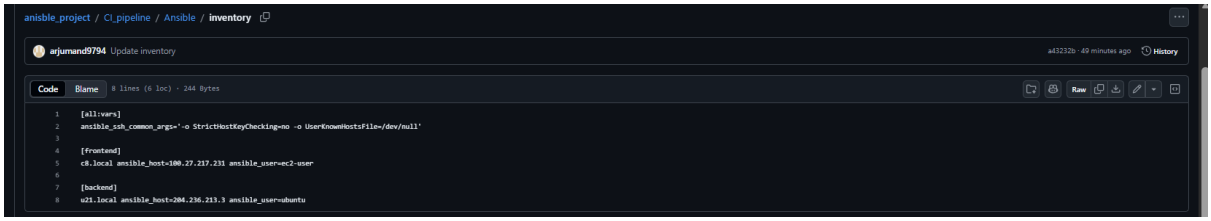
5 Ansible Configuration Explained

- inventory

[all:vars]

ansible_ssh_common_args='-o StrictHostKeyChecking=no'

✔ Prevents SSH host key verification failure in Jenkins



- **site.yml (Main Playbook)**

- hosts: all

  become: yes

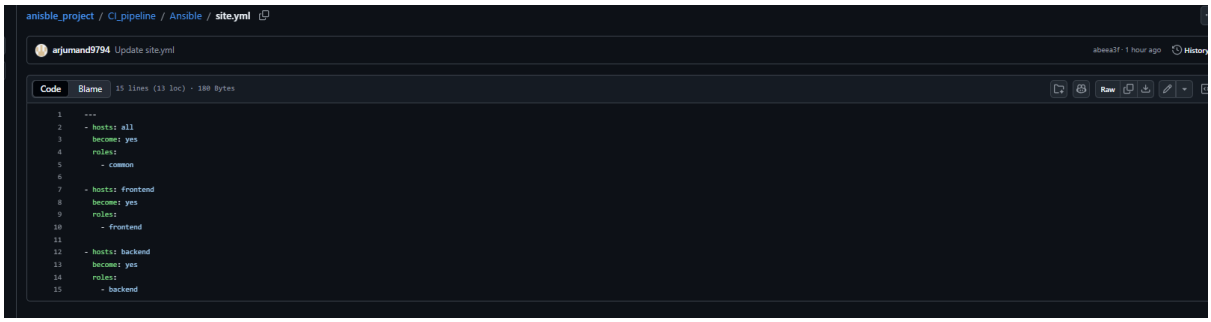
  roles:

    - common

- hosts: frontend

```
  become: yes
  roles:
    - frontend

- hosts: backend
  become: yes
  roles:
    - backend
```

✔ Executes roles in correct order
✔ Ensures OS configs before app setup



6️⃣ Ansible Roles Explained

🔹 Common Role (roles/common)

Purpose: Base OS configuration

Tasks:

- Disable SELinux (Amazon Linux)
- Stop firewalld if present
- Stop UFW on Ubuntu

✔ Uses OS-specific conditions

✔ Prevents service-related failures

🔹 Frontend Role (roles/frontend)

Purpose: Configure Nginx as reverse proxy

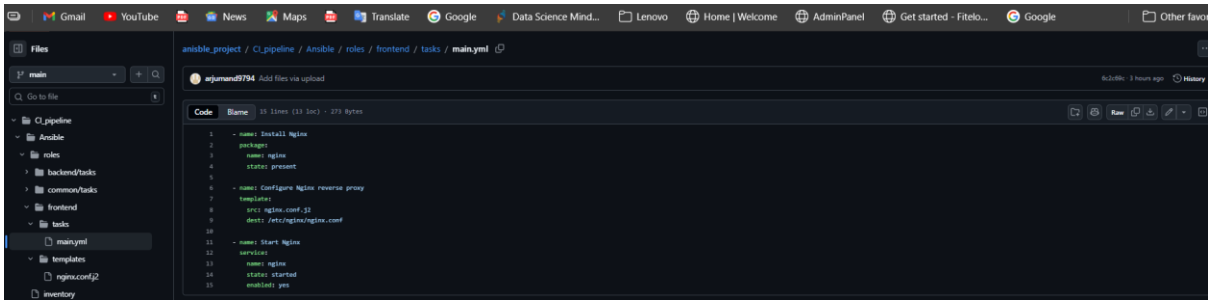
Tasks:

- Install Nginx
- Apply reverse proxy configuration
- Start & enable Nginx service

nginx.conf.j2

```
upstream backend_netdata {
  server {{ hostvars[groups['backend'][0]].ansible_host }}:19999;
}
```

✔ Routes traffic from port 80 → 19999



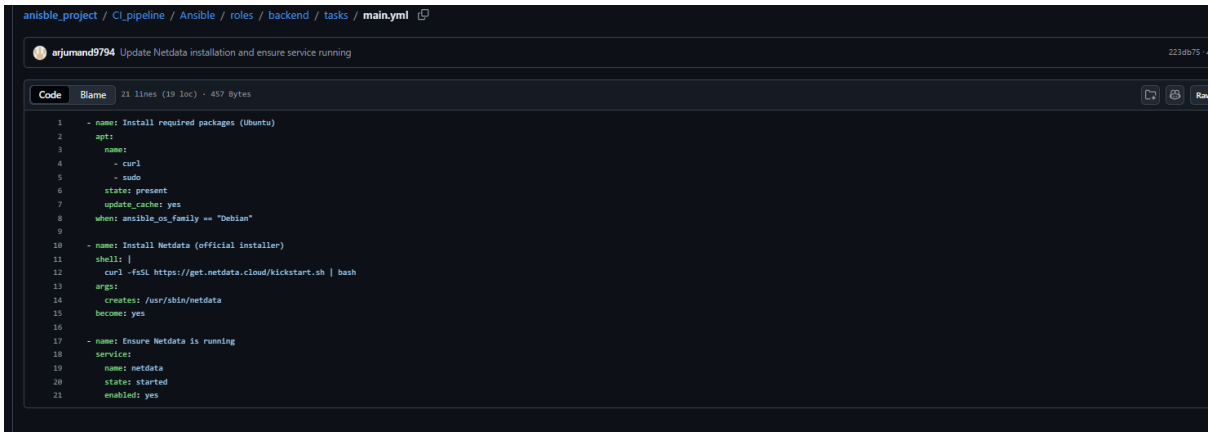
🔹 Backend Role (roles/backend)

Purpose: Install Netdata monitoring

Tasks:

- Install Netdata using official script
- Ensure Netdata service is running

✔ Netdata runs on port 19999



7️⃣ Jenkins Pipeline Explained

🔹 Jenkinsfile Stages

Stage	Purpose
Checkout	Pull latest GitHub code
Terraform Init	Initialize Terraform
Terraform Apply	Provision infrastructure
Ansible Configure	Configure EC2 instances

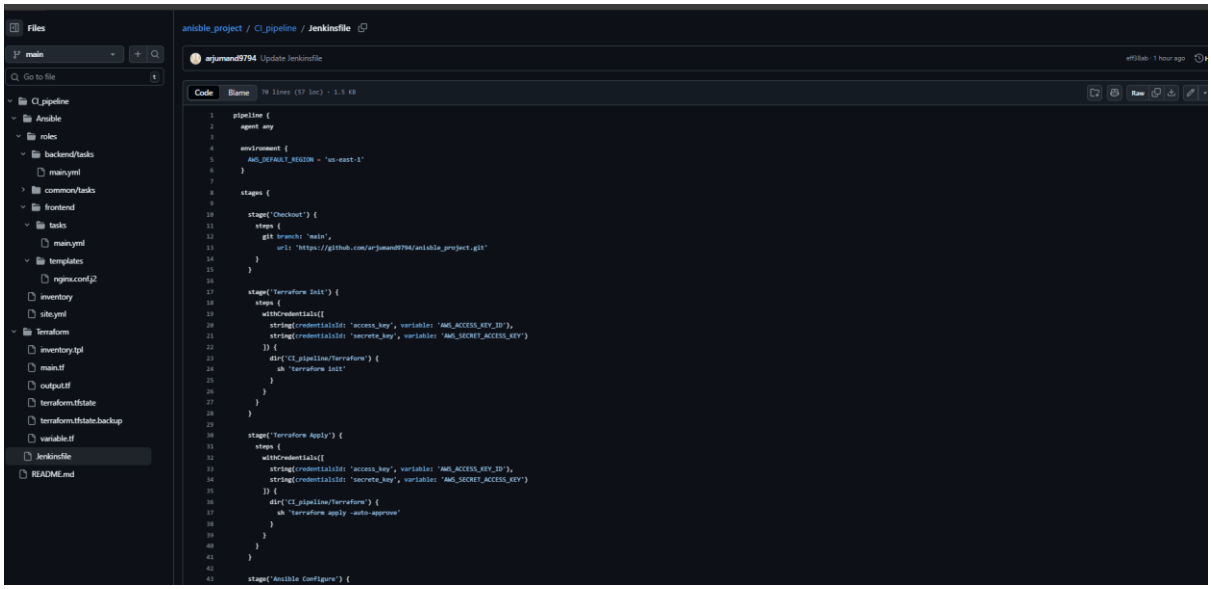
🔹 Jenkinsfile (Final Version)

```
pipeline {
  agent any


  environment {
    AWS_DEFAULT_REGION = 'us-east-1'
  }
```

```
  stages {

    stage('Checkout') {
      steps {
        git branch: 'main',
            url: 'https://github.com/arjumand9794/anisble_project.git'
      }
    }

    stage('Terraform Init') {
      steps {
        withCredentials([
          string(credentialsId: 'access_key', variable: 'AWS_ACCESS_KEY_ID'),
          string(credentialsId: 'secrete_key', variable:
'AWS_SECRET_ACCESS_KEY')
        ]) {
          dir('CI_pipeline/Terraform') {
            sh 'terraform init'
          }
        }
      }
    }
    stage('Terraform Apply') {
```

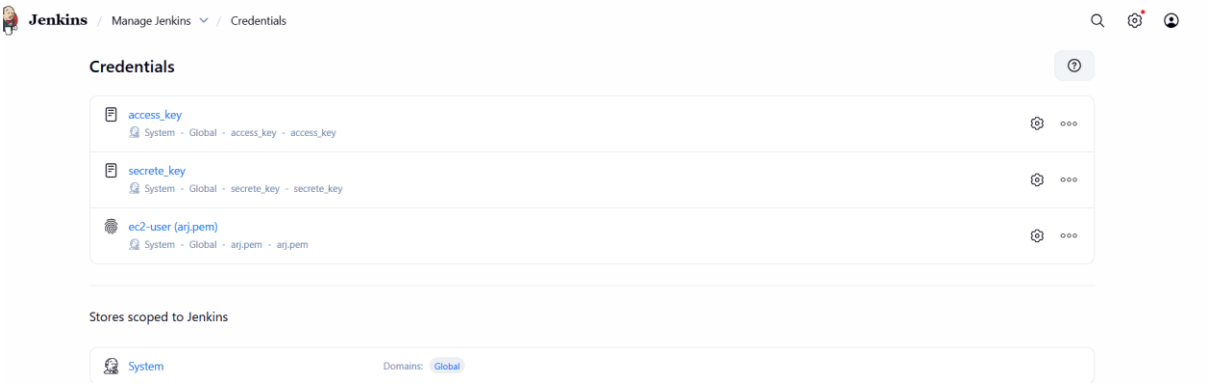
```
      steps {
        withCredentials([
          string(credentialsId: 'access_key', variable: 'AWS_ACCESS_KEY_ID'),
          string(credentialsId: 'secrete_key', variable:
'AWS_SECRET_ACCESS_KEY')
        ]) {
          dir('CI_pipeline/Terraform') {
            sh 'terraform apply -auto-approve'
          }
        }
      }
    }

    stage('Ansible Configure') {
      steps {
        withCredentials([
          sshUserPrivateKey(
            credentialsId: 'arj.pem',
            keyFileVariable: 'SSH_KEY',
            usernameVariable: 'SSH_USER'
          )
        ]) {
          dir('CI_pipeline/Ansible') {
            sh '''
```

```
            chmod 600 $SSH_KEY
            ansible-playbook -i inventory site.yml \
              --private-key $SSH_KEY
          '''
        }
      }
    }
  }
}
```



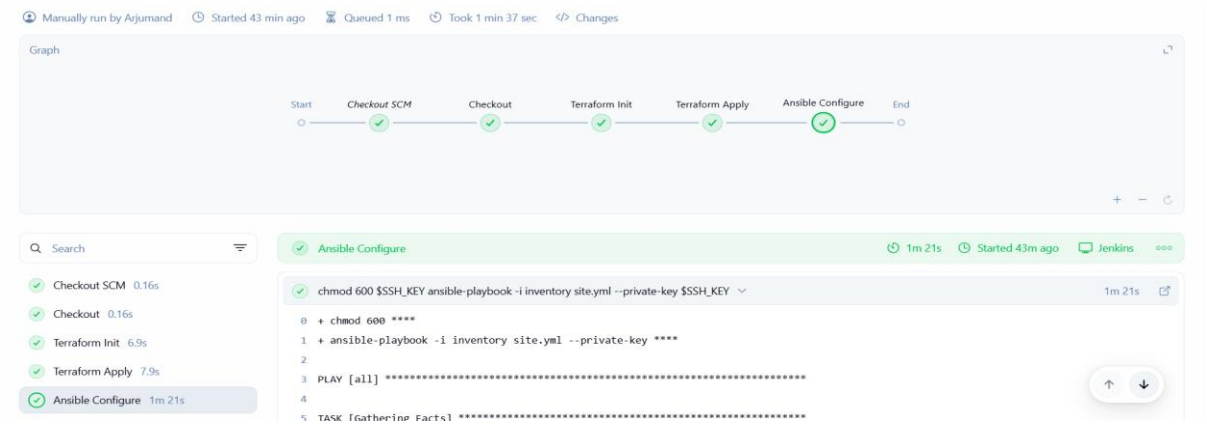
→Add credentials ssh_key type of credentials in global and add pem key and give as ec2-user

→Add secrete key and access separately with secrete text as type in global credentials

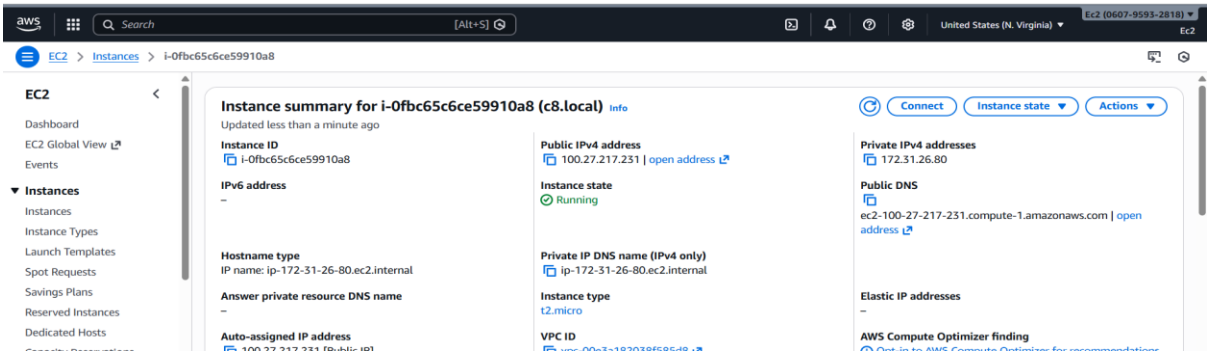


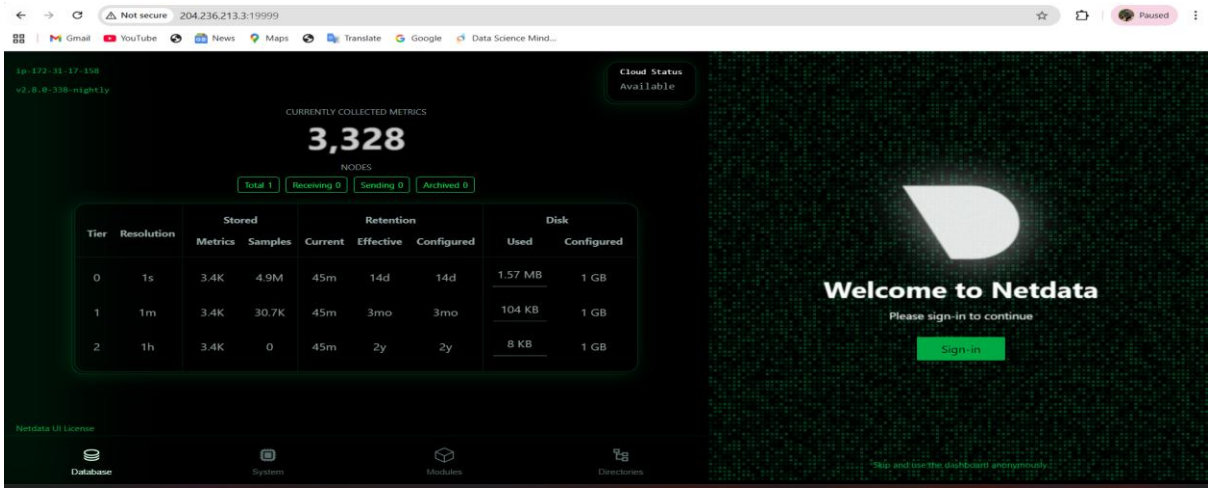
8 Final Result

- Terraform creates infrastructure
- Inventory generated automatically
- Ansible configures:
  - Frontend → Nginx reverse proxy
  - Backend → Netdata monitoring
- Jenkins pipeline completes successfully

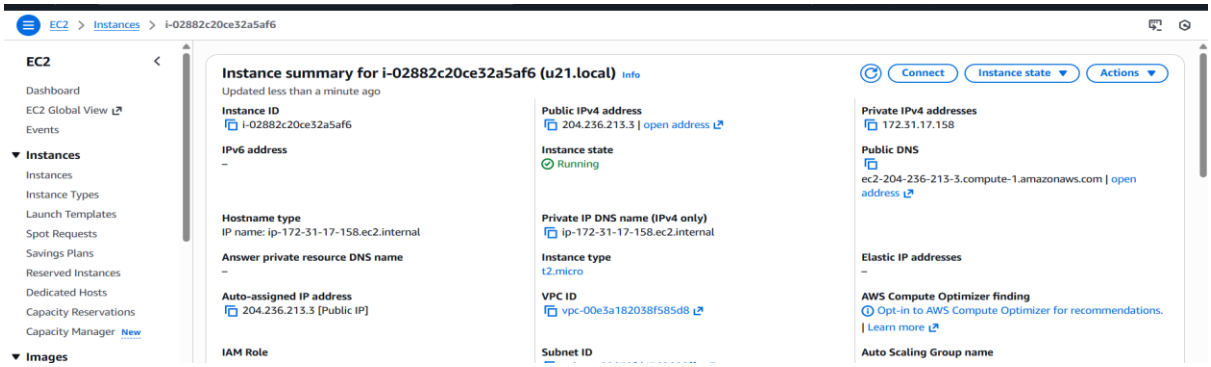


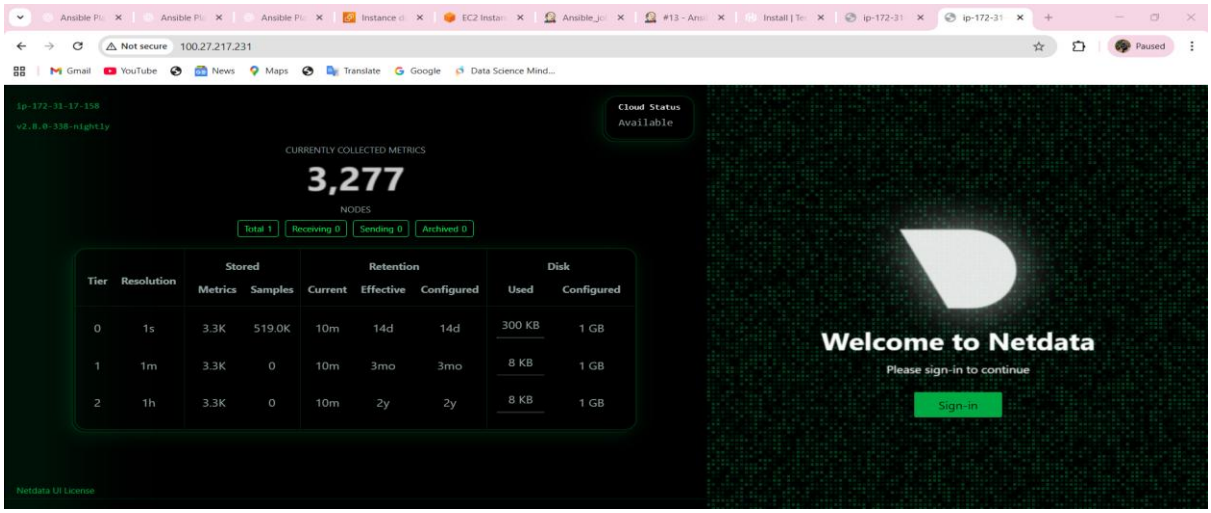
→ c8.local with public Id routed to 19999





→ u21.local with public ip routed to 19999 port number





9 Key Skills Demonstrated

- ✔ Infrastructure as Code
- ✔ CI/CD pipeline design
- ✔ Configuration management
- ✔ AWS automation
- ✔ Real-world troubleshooting